

Sherlock's Last Case PRD

Consolidated, complete specification

1. Overview

A browser-based, team-play detective game for a techfest. Teams of up to 3 (1 head + up to 2 investigators, each on their own computer) play one of 10 fixed mystery cases, querying a live read-only Postgres database through an in-browser SQL console to uncover clues across 3 phases, culminating in naming the culprit. Built on Next.js + Supabase, deployed on Vercel.

Scale: ~30-35 concurrent computers, ~10-12 teams, one live event session (not an ongoing product).

Target session length: 15-20 min average solver, ~10 min for strong SQL users.

2. Goals / Non-Goals

Goals

- Reliable, glitch-free play for ~35 concurrent clients on event day.
- Zero cross-team data leakage (a team can never see another team's or another case's data).
- Deterministic, tamper-proof grading — no ambiguity in what counts as "correct."
- A live leaderboard the room can watch.
- Graceful recovery if a browser crashes or a computer needs restarting mid-game.
- Query execution that is safe even if every other safeguard fails (defense in depth).

Non-Goals

- No persistent user accounts beyond the event; sessions are ephemeral.
- No support for more than 10 simultaneous case templates.
- No AI-based or fuzzy-text grading anywhere in the system.

- No cross-event history, replay, or persistent player profiles.

3. Core Concepts & Terminology

Term	Meaning
Case	One of 10 fixed mystery templates — the puzzle itself
Team	One group's play-through of a case; created when the head joins
Case Code / Case Password	Credentials identifying <i>which puzzle</i>
Team Code	Generated on team creation; identifies <i>this team's session</i> , used by investigators to join
Phase	One of 3 stages within a case, each unlocking more tables/characters
Task	One graded question within a phase (3 per phase: easy/medium/hard)
Final Deduction	The culprit submission, unlocked after Phase 3 clears

4. Functional Requirements

4.1 Case & team access

- A case has a public `case_id` + `case_password`. These identify the **puzzle**, not a team.
- **First join (head)**: enters `case_id + case_password` → server creates a new `team` row scoped to that case, generates a unique **team_code** (e.g. `MAPLE-482`), displays it to the head to share with teammates. Player role = `head`.
- **Subsequent joins (investigators)**: enter the **team_code** only (not the case password) → joins that exact team. Role = `investigator`.
- A 4th join attempt on a full team is rejected with a clear message.
- Multiple teams *can* play the same case template (if more than 10 teams exist); each gets its own isolated `team_code` and its own instance of game state, though they share the same underlying case content/answers. For a room of ~10 teams this likely won't happen, but the model supports it cleanly.

- `UNIQUE(team_code)` enforced at the DB level.
- `UNIQUE(team_id) WHERE role = 'head'` — only one head per team, enforced at the DB level.

4.2 Team roles

- **Head:** sees a task board of all currently-open tasks across the active phase, can assign any open task to self or either investigator. Can reassign before a task is answered.
- **Investigator:** sees their assigned task, and can self-serve any unassigned open task if idle.
- All 3 players see phase narrative, unlocked tables, and team progress in sync (realtime).

4.3 Game flow / state machine

Server-owned state, never client-settable:

```
WAITING → PHASE_1 → PHASE_2 → PHASE_3 → FINAL_DEDUCTION → FINISHED
```

- `WAITING`: team created, head may wait briefly for teammates or start immediately (configurable — recommend allowing immediate start so a 1-2 person team isn't blocked).
- Transitioning `PHASE_n → PHASE_n+1` happens **only** as a server-side side effect of all 3 of that phase's tasks being answered correctly — never via a client request setting `current_phase` directly.
- Each phase reveal (narrative text, newly unlocked tables/characters) is delivered via the phase-clear response and broadcast to all 3 teammates via realtime.
- Tables unlocked in a phase remain queryable in every later phase (strictly additive, never revoked).
- `FINAL_DEDUCTION` unlocks a single culprit dropdown (full character roster for that case). One submission per team (`UNIQUE(team_id)` on `culprit_submission`). Submitting transitions to `FINISHED` and stamps `finished_at` .

4.4 Task design

- Exactly 3 tasks per phase: 1 easy, 1 medium, 1 hard.
- Tasks within a phase are **dependent/chained** where possible — task 1 narrows the suspect pool, task 2 narrows further using a new table, task 3 nearly isolates the answer — rather than 3 unrelated questions.
- Each task has an **answer_type**, which determines both the UI input and the grading mechanism:

answer_type	UI	Grading
<code>SINGLE_CHARACTER</code>	Dropdown of case roster	Exact match on character_id
<code>NUMBER</code>	Numeric input	Exact match
<code>TIME</code>	Time picker (canonical HH:MM)	Exact match
<code>TEXT</code>	Avoided wherever possible; if used, a single canonical short string we author ourselves (e.g. a table/column-derived code), never open-ended enough to vary by phrasing	Exact match, case-insensitive, trimmed

- No free-text grading of anything requiring natural-language interpretation, anywhere in the system (culprit, motive, evidence — none of these are separately graded as open text).
- Every case includes 2-3 intentional red herrings — people/clues who look suspicious but are cleared by the data — documented explicitly in that case's authoring spec.

4.5 Scoring

Component	Points
Phase 1 cleared	20
Phase 2 cleared	20
Phase 3 cleared	20
Correct culprit	40
Total	100

- Leaderboard sort: `total_points DESC, (finished_at - started_at) ASC` — time is a tiebreaker only, never folded into the point total, so speed never outranks correctness.
- All scoring operations are idempotent (see 6.6).

4.6 Query console

- Each player gets a SQL editor, a "Run" button, and a results grid.
- Scoped to only the tables their team has unlocked so far (enforced server-side via RLS, not just hidden in the UI).
- Read-only: `SELECT` only.
- Results capped (e.g. 200 rows) with a visible "truncated" notice if exceeded.
- Lightweight syntax pre-check (must start with `SELECT`, single statement, no semicolons) — **this is a UX convenience for a friendly error message, not a security boundary** (see 6.1 for why).

4.7 Session recovery

- If a computer crashes, locks up, or a browser tab is closed, rejoining with the team_code (or case credentials, for the head) and the same role restores exact current state — phase, completed tasks, assignments, scores — all derived server-side, never from client-cached state.

4.8 Admin / organizer tools

v1 scope:

- Live dashboard: `Case | Team | Phase | Elapsed time | Status` for every active team.
- Reset a single team's state (without touching any other team).
- Inspect a team's submitted answers and recent query errors.
- Emergency pause (freezes timers/scoring without losing state).
- Export leaderboard.

Deferred (not v1): force-advance phase, disable/reopen individual cases mid-event, per-query live tailing.

5. Non-Functional Requirements

5.1 Scale reality check

30-35 browser clients and ~10-12 teams is light load for both Postgres and Vercel — this is not a raw-throughput problem. The actual risks are: one

student running an expensive or malformed query, all 3 teammates hammering "Run" simultaneously, and simultaneous phase-clear events needing to reach 3 teammates instantly. Design priority is **correctness and abuse-resistance**, not horizontal scale.

5.2 Availability target

Must run correctly, unattended-except-for-admin, for a continuous ~1-2 hour event window without restart.

5.3 Performance targets

- Query console response: under ~1s for typical case-sized tables (expect tens to low hundreds of rows per table).
- Realtime phase-unlock propagation to teammates: under ~1s.
- Page load / session restore after reconnect: under ~2s.

6. Technical Architecture

6.1 Security model — layered, defense in depth

```
Layer 1 (load-bearing): dedicated Postgres role for query execution – SELECT-only grants, zero write access
Layer 2 (load-bearing): RLS on every data table, policy keyed on the session's case_id
Layer 3 (load-bearing): statement_timeout (3-5s) on that role
Layer 4 (load-bearing): enforced row cap / auto-appended LIMIT
Layer 5 (load-bearing): per-player rate limit on /api/query
Layer 6 (UX only):      "must start with SELECT" syntax pre-check → friendly error message
```

Principle: if Layer 6 lets something unanticipated through, Layers 1-5 still make it harmless. No individual check is trusted as the sole safeguard.

Critical rule: the role used for `/api/query` execution is a normal RLS-subject role — **never** the Supabase service-role key, which bypasses RLS entirely. This is called out explicitly in the build checklist so the service key is never reached for out of dev-time convenience and accidentally shipped.

6.2 Authentication / session model

Server is sole authority for all identity and progress state.

```
Browser → POST /api/join (case creds, or team_code)
Next.js API validates → creates/finds team + player row
                        → issues signed session token (team_id, player_id, role, case_id)
Browser stores token → sent with every subsequent request
```

Every later request (`/api/query` , `/api/submit-task` , `/api/submit-culprit`) re-derives `team_id` / `role` / `case_id` **from the server-side session**, never from any client-supplied field.

6.3 Team joining logic (explicit)

```
First join:
  case_id + case_password → validate → create team row
                              → generate unique team_code
                              → player.role = 'head'

Subsequent joins:
  team_code → find team → check player count < 3
                              → player.role = 'investigator'
                              → reject if team already has 3 players
```

6.4 RLS architecture

```
session → team_id → teams.case_id → RLS policy: data_row.case_id = teams.case_id
```

Every case-specific clue table (`cast_crew`, `access_log`, `text_messages`, etc.) carries a `case_id` column and an RLS policy restricting visible rows to the requesting session's case.

6.5 Query execution path

```
Browser → POST /api/query { sql_text }
Next.js:
  1. authenticate session (team_id, case_id, role)
  2. resolve which tables are currently unlocked for this team's phase
  3. syntax pre-check (UX layer – see 6.1)
  4. forward to Postgres via the pooled connection, using the read-only role
Postgres:
  5. RLS filters rows to this case_id
  6. statement_timeout bounds execution time
```

```
7. row cap / enforced LIMIT bounds result size
8. results returned → logged (see 6.9) → sent to browser
```

- Connections go through Supabase's **pooled (transaction-mode) connection string**, not direct Postgres connections from the browser.
- Database connection strings and the Supabase service-role key live only in server-side environment variables — never exposed to the client bundle.

6.6 Idempotency & transactional integrity

- `task_submissions`: unique constraint on `(task_id, team_id)` for the first *correct* submission — a duplicate correct submission is a no-op, not double points.
- `culprit_submission`: `UNIQUE(team_id)` — one shot, enforced at the DB level.
- Phase-clear detection, score update, and state transition happen inside the same transaction as the triggering submission, so a retried/duplicate request can't partially apply.
- Team creation (`/api/join` for a head) is similarly guarded against double-submission (e.g. double-click) creating two teams for one case_password entry within a short window.

6.7 Realtime with fallback

```
Page load → GET authoritative current state (phase, scores, assignments, unlocked tables)
              → subscribe to this team's realtime channel
Realtime event arrives → merge into local state
WebSocket disconnects → reconnect → re-GET authoritative state → resume
```

Realtime is a convenience layer for instant updates across teammates; correctness never depends on it arriving — every client can always fall back to polling/re-fetching authoritative server state.

One Supabase Realtime channel per team (`team_id` -scoped), broadcasting: phase-unlock events, task-completion status, player presence.

6.8 Deployment

```
Next.js (App Router, API routes) → Vercel
Supabase (Postgres, RLS, Realtime, pooled connections)
```

Keep the backend thin — Next.js API routes handle validation/session/orchestration; Postgres does the heavy lifting via RLS + roles rather than application-layer permission checks.

6.9 Logging

```
query_logs(id, team_id, player_id, query_hash, execution_time_ms, row_count, success,
error_type, created_at)
```

Sufficient to debug a live issue during the event ("why did Team 7's console stop responding") without guesswork. Full SQL text retained only if useful for debugging during development; can be hashed/truncated for the live event if preferred.

7. Data Model

```
cases(id, password_hash, title, status, hook_text)
case_phases(case_id, phase_number, clue_text, tables_unlocked text[])
tasks(id, case_id, phase_number, difficulty, prompt_text, answer_type, correct_answer,
sql_concept)
characters(id, case_id, name, role_in_story)          -- powers the culprit dropdown

teams(id, case_id, team_code UNIQUE, team_name, started_at, finished_at)
players(id, team_id, display_name, role)              -- role: head | investigator
                                                    -- UNIQUE(team_id) WHERE role
                                                    -- = 'head'

task_submissions(id, task_id, team_id, player_id, submitted_value, is_correct, submitte
ed_at)
                                                    -- UNIQUE(task_id, team_id) for
correct submissions
team_progress(team_id, current_phase, phase1_cleared, phase2_cleared, phase3_cleared)
team_scores(team_id, phase1_pts, phase2_pts, phase3_pts, culprit_pts, total_pts)
culprit_submission(team_id UNIQUE, chosen_character_id, is_correct, submitted_at)

query_logs(id, team_id, player_id, query_hash, execution_time_ms, row_count, success,
error_type, created_at)

-- Per-case clue data tables (cast_crew, access_log, prop_handling_log, text_messages,
etc.)
-- each row carries case_id, RLS-filtered per the requesting team's assigned case
```

ER structure (text form):

```

CASES
├─ CASE_PHASES
├─ TASKS
├─ CHARACTERS
├─ <clue data tables> (access_log, text_messages, ...)
└─ TEAMS
    ├─ PLAYERS
    ├─ TASK_SUBMISSIONS
    ├─ TEAM_PROGRESS
    ├─ TEAM_SCORES
    └─ CULPRIT_SUBMISSION

```

8. API Surface

Endpoint	Purpose
<code>POST /api/join</code>	case_id+password (head) or team_code (investigator) → session token
<code>POST /api/query</code>	Execute a read-only SQL query, scoped and validated
<code>POST /api/submit-task</code>	Submit an answer for a task; triggers phase-clear + scoring if applicable
<code>POST /api/submit-culprit</code>	Final culprit submission; finalizes team, updates leaderboard
<code>GET /api/state</code>	Authoritative current state for session restore (phase, scores, assignments)
<code>GET /api/leaderboard</code>	Live standings (also realtime-subscribed)
<code>POST /api/admin/reset-team</code>	Admin-only: reset one team's state
<code>GET /api/admin/dashboard</code>	Admin-only: live view of all teams/cases

9. Case Authoring Specification

Every one of the 10 cases is written against this template:

```

CASE ID / TITLE

SETTING / CRIME / CULPRIT / MOTIVE / METHOD / TIMELINE (ground truth, not shown directly)

CHARACTERS (10-15, all with personality, not just suspects)

RED HERRINGS (2-3, explicitly noted)

```

```
PHASE 1
  Tables unlocked:
  Clue (narrative, multi-reveal):
  Task 1 (easy):      answer_type, correct_answer
  Task 2 (medium):   answer_type, correct_answer
  Task 3 (hard):     answer_type, correct_answer
  SQL concepts used:

PHASE 2 – same shape, tables accumulate, baseline difficulty just past Phase 1
PHASE 3 – same shape, tables accumulate, noticeably tougher

FINAL DEDUCTION
  Culprit (character_id)

ESTIMATED SOLVE TIME
  Beginner / Average / Strong SQL user

VALIDATION CHECKLIST
- Can the culprit be uniquely determined from only the data unlocked by Phase 3?
- Can every task be solved using only tables unlocked at that point?
- Is there an accidental alternative solution the data also supports?
- Checked by someone who did not know the intended answer in advance?
```

10. Testing & Rollout Plan

1. **Build against one fully-specified case** (Opening Night) end-to-end: join flow, all 3 phases, query console, scoring, culprit dropdown, admin reset.
2. **Load-test** `/api/query` with ~35 simulated concurrent clients to confirm pooled connections + rate limiting + `statement_timeout` hold under realistic (light) load.
3. **Adversarial query testing**: deliberately attempt expensive/malformed/cross-table queries against the read-only role to confirm Layers 1-5 hold even when Layer 6 is bypassed.
4. **Crash/recovery testing**: kill a browser mid-phase, confirm rejoin via `team_code` restores exact state.
5. **Dry run** with a small group playing the finished reference case to surface UX friction before the other 9 are authored.
6. **Independent validation** of each of the 10 cases by someone without knowledge of the intended solution, against the validation checklist in Section 9.

7. **Event-day runbook:** admin dashboard open and monitored throughout; reset procedure rehearsed in advance.
-

11. Other Decisions

- Exact team size floor: can a team of 1 or 2 play (e.g. head with no investigators joining), or is 3 required to start?
- What happens to an abandoned team (joined, never progressed) — does it block its team_code/case slot for the rest of the event?
- Whether query_logs retains full SQL text or only a hash during the live event (privacy/debugging tradeoff — low stakes here, but worth deciding explicitly).
- Tie-break beyond time (if two teams finish with identical points and identical time, which is unlikely but possible).

1–3 players allowed, because you don't want an event-day connectivity problem to prevent someone from playing.

Abandoned team: admin can reset it.

SQL logs: retain them during the event for debugging, then discard them after the event.

Tie: whoever submitted the final correct culprit first, using server timestamp.

Now, make me a phase by phase work laid out, in detail. So, that we can complete one phase at a time and test it thoroughly

Build Roadmap — Phase by Phase

To avoid confusion with the game's own Phase 1/2/3, I'll call these **Build Stages**. Each stage is meant to be built, tested, and confirmed working in isolation before moving to the next — so a bug never has more than one stage's worth of surface area to hide in.

Build Stage 0 — Foundations

Goal: empty skeleton that deploys and connects, nothing playable yet.

Build:

- Next.js app (App Router) scaffolded, deployed to Vercel.
- Supabase project created; pooled connection string configured server-side only.
- Core schema created (no RLS yet): `cases`, `teams`, `players`, `case_phases`, `tasks`, `characters`, `task_submissions`, `team_progress`, `team_scores`, `culprit_submission`, `query_logs`.
- Environment variables set (service key and DB connection string both server-side only — never in client bundle).
- One dummy case row inserted manually for testing.

Test before moving on:

- App deploys and loads a blank page on Vercel.
- Can read/write the dummy case row from a server-side script (confirms connection + pooling works).
- Confirm (by inspecting the client bundle) that no secret key or connection string leaked to the browser.

Build Stage 1 — Join & Session

Goal: a head can create a team, investigators can join it, and the server knows who's who — nothing about gameplay yet.

Build:

- `POST /api/join`: `case_id+password` → creates team, generates `team_code`, issues session token (head path).
- `POST /api/join`: `team_code` → joins existing team as investigator, rejects if team is full (3) — but remember 1-3 is valid, so "full" only blocks a 4th.
- Session token issuance + validation middleware for subsequent requests.
- `UNIQUE(team_code)`, `UNIQUE(team_id) WHERE role='head'` constraints in place.
- Minimal UI: a join screen (case creds / team code), and a post-join screen showing role + `team_code` (for the head to share).

Test before moving on:

- Head joins with valid case creds → team created, `team_code` generated and displayed.

- Second/third player joins with team_code → both become investigators, team now shows 2-3 players.
- 4th join attempt → clean rejection.
- Wrong case password → clean rejection, no team created.
- Reusing a team_code after closing the browser and rejoining → resumes the same session (role preserved), doesn't create a duplicate player.
- Double-click/double-submit on the head's join request → confirm only one team is created, not two.

Build Stage 2 — State Machine & Game Phases (no query console yet)

Goal: phases advance correctly server-side, with hardcoded/stubbed task answers — proving the state machine before wiring in real SQL.

Build:

- `case_phases` and `tasks` populated for one dummy case (simple stub tasks, e.g. "type the number 5").
- `POST /api/submit-task`: validates against `answer_type`, records in `task_submissions`, checks if all 3 tasks in the phase are now correct, advances `team_progress.current_phase` if so.
- `GET /api/state`: returns current phase, which tables/characters are "unlocked" (stubbed), task list with assignment/status.
- Idempotency: resubmitting an already-correct task is a no-op, doesn't re-score.
- Minimal UI: phase narrative text, task list with input per `answer_type`, submit buttons.

Test before moving on:

- Submitting all 3 Phase 1 tasks correctly advances to Phase 2 — confirmed via `/api/state`, not just the UI.
- Submitting a wrong answer does not advance anything, no points awarded.
- Resubmitting a correct task doesn't double-score.

- Attempting to directly manipulate state (e.g. crafting a request implying Phase 3) is rejected — confirms server, not client, owns `current_phase`.
- Two teammates submitting different tasks for the same phase simultaneously both register correctly and the phase advances exactly once.

Build Stage 3 — Query Console & Real Case Data

Goal: one real case's data tables exist, RLS is enforced, and the query console actually works safely.

Build:

- Real per-case clue tables for the reference case (e.g. Opening Night) created with `case_id` columns.
- Dedicated read-only Postgres role created, granted `SELECT` only.
- RLS policies on every clue table keyed on `case_id`.
- `POST /api/query` : authenticates session → checks table is unlocked for current phase → syntax pre-check (UX layer) → executes via pooled connection under the read-only role → `statement_timeout` + row cap applied → logs to `query_logs`.
- Per-player rate limit on `/api/query`.
- Query console UI: SQL editor + run button + results grid + "truncated" notice when capped.
- Replace Stage 2's stubbed tasks with real tasks referencing this case's actual tables and `correct_answer` values.

Test before moving on (this is the security-critical stage — test thoroughly):

- A team can query only tables unlocked at their current phase; querying a not-yet-unlocked table is rejected or returns nothing.
- A team cannot see another case's data even if they guess a table name (confirm via RLS, not just app logic — try hitting `/api/query` directly with a crafted request bypassing the UI).
- Attempt a deliberately expensive query (large cross join) — confirm `statement_timeout` kills it cleanly rather than hanging the connection pool.

- Attempt `INSERT` / `UPDATE` / `DELETE` / `DROP` through the console — confirm the role's lack of grants blocks it even if the syntax pre-check were somehow bypassed.
 - Rate limit triggers correctly on rapid repeated submissions.
 - Real tasks for this one case are solvable end-to-end using only the console, and `correct_answer` grading works for each `answer_type` used.
-

Build Stage 4 — Scoring, Leaderboard & Culprit Submission

Goal: full scoring loop and the final deduction step.

Build:

- Score updates wired into `/api/submit-task` (20 pts per phase cleared) and a new `/api/submit-culprit` (40 pts, `UNIQUE(team_id)`).
- `culprit_submission` unlocks only after Phase 3 is cleared; dropdown populated from `characters` .
- `finished_at` stamped on culprit submission; `started_at` stamped at team creation (Stage 1).
- `GET /api/leaderboard` : sorted by `total_points DESC` , tie-break by earliest correct culprit submission timestamp.
- Minimal leaderboard UI.

Test before moving on:

- Full case playthrough end-to-end: join → Phase 1 → Phase 2 → Phase 3 → culprit submission → correct score (100) and correct timestamp recorded.
 - Submitting the culprit before Phase 3 clears is rejected.
 - Resubmitting a culprit after already submitting is rejected (no double submission, no score change).
 - Two teams finishing with identical scores — tie-break resolves correctly by earliest correct submission timestamp.
 - Wrong culprit submission — team gets 0 culprit points, `finished_at` still stamped (they only get one shot, per the decision).
-

Build Stage 5 — Realtime Sync

Goal: all 3 teammates see phase unlocks and task completions live, with fallback if the socket drops.

Build:

- One Supabase Realtime channel per team, broadcasting: phase-unlock, task-completion, presence.
- Client: subscribe on load after fetching authoritative `/api/state`; merge realtime events into local state.
- Reconnect logic: on socket drop, reconnect and re-fetch `/api/state` rather than trusting stale local state.
- Head's task-assignment UI reflects investigator actions live.

Test before moving on:

- Player A completes a task; Players B and C see the update within ~1s without refreshing.
 - Kill the WebSocket connection deliberately (e.g. devtools network throttling/offline toggle) mid-game, then restore it — client reconnects and state is correct, not stale or duplicated.
 - All 3 teammates see the same phase-unlock narrative at the same moment when the phase clears.
-

Build Stage 6 — Recovery & Admin Tools

Goal: the event can survive a crashed computer, and organizers have visibility/control.

Build:

- Rejoin flow (Stage 1's join endpoint, reused) correctly restores full state after a closed/crashed browser — confirm this explicitly, it was designed for in Stage 1 but needs end-to-end verification now that there's real state to restore.
- Admin auth (simple password-gated route).
- `GET /api/admin/dashboard`: live table of all teams — case, phase, elapsed time, status.

- `POST /api/admin/reset-team` : clears one team's state without affecting others.
- Admin view of a team's submitted answers and recent query errors (from `query_logs`).
- Emergency pause (freeze timer/scoring for a team or globally).

Test before moving on:

- Mid-game browser crash/close → rejoin with `team_code` → exact state restored (phase, scores, assignments).
- Admin resets an abandoned team → that team's slot is fully clean and rejoinable; no effect on other teams' data or scores.
- Admin dashboard reflects live status within the event's expected refresh window.

Build Stage 7 — Load & Adversarial Testing

Goal: confirm the system holds under realistic event conditions and deliberate misuse, before content is scaled up.

Build: no new features — test harness only.

Test:

- Simulate ~35 concurrent clients hitting `/api/query` and `/api/submit-task` — confirm pooled connections, rate limiting, and `statement_timeout` behave correctly under concurrent load.
- Adversarial pass: attempt SQL injection-style payloads, attempt to read another case's tables directly, attempt to replay/forge a session token, attempt rapid double-submission on every scoring endpoint.
- Confirm `query_logs` captures enough detail to debug a hypothetical "Team 7's console stopped working" scenario without guessing.

Build Stage 8 — Remaining Case Content

Goal: scale from 1 proven case to all 10.

Build:

- Author the remaining 9 cases against the Case Authoring Specification (Section 9 of the PRD) — unique settings, varied motives, dependent task

chains, red herrings, typed answers.

- Each case independently validated by someone who doesn't know the intended solution (per the checklist).

Test:

- Each of the 10 cases is playable end-to-end solo by a tester, within the target time range (beginner/average/strong).
 - Spot-check RLS isolation across a few different cases simultaneously (not just the one case tested in earlier stages).
-

Build Stage 9 — Event-Day Rehearsal

Goal: full dress rehearsal under near-real conditions.

Build: none — operational readiness only.

Test:

- All 10 cases assigned to simulated teams, played concurrently on separate machines/browsers if possible.
 - Admin dashboard monitored throughout; reset and pause rehearsed at least once.
 - Confirm leaderboard, timing, and tie-break behavior with real concurrent finishers.
 - Post-event: confirm `query_logs` can be cleanly discarded as decided.
-